

Pointer -

Loops -

Recursion -

Class and object -

Static variable

- If declared inside a function, its scope remains local to that function, but its value persists between function calls.
- We can change its value
- If a static variable is not explicitly initialized, it is initialized to zero.
- static gives the variable internal linkage

const variable

- A const object can be read but not modified

#	Details	Particular
	<pre>main() { char *p; p = "Hello"; printf("%cn", *&p); }</pre>	H
	<pre>main() { char *p; printf("%d %d", sizeof(*p), sizeof(p)); }</pre>	1 2
	<pre>int main() { int i, j, count; count = 0; i = 0; for (j = -3; j <= 3; j++)</pre>	10

	<pre> { if ((j >= 0) && (i++)) { count = count + j; } } count = count + i; printf("%d", count); return 0; } </pre>	
	<pre> char inchar = 'A'; switch (inchar) { case 'A' : printf ("choice A \n") ; case 'B' : printf ("choice B ") ; case 'C' : case 'D' : case 'E' : default: printf ("No Choice") ; } </pre>	Choice A Choice B No choice
	<pre> void main() { int *ptr, a=10; ptr = &a; *ptr += 1; printf("%d, %d", *ptr, a); } </pre>	11
	<pre> void main() { int i=3, *j, **k; j = &i; k = &j; printf("%d%d%d", *j, **k, *(*k)); } </pre>	333
	<pre> void main() { int array[10]; int *i = &array[2], *j = &array[5]; int diff = j-i; printf("%d", diff); } </pre>	3
	<pre> int main() { int m = 10; } </pre>	

<pre> int n, n1; n = ++m; n1 = m++; n--; --n1; n -= n1; printf("%d",n); return 0; } </pre>	

Program List -

1. Armstrong number
2. Reverse a string using recursion
3. Finding prime number
4. Printing different patterns
- 5.

Criteria	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
Primary Focus	Focuses on the process or step-by-step instructions to complete a task.	Focuses on data and the real-world objects that interact with it.
Program Division	The main program is divided into smaller parts called functions .	The program is divided into smaller parts called objects .
Approach	Follows a top-down approach (starting from the main goal and breaking it down).	Follows a bottom-up approach (building objects first, then combining them).
Data Security	Low security; data is often global and can move freely between functions.	High security; encapsulation and data hiding protect the data.
Access Specifiers	No access specifiers are used; everything is generally accessible.	Uses access specifiers like public, private, and protected .
Inheritance	Does not support the concept of inheritance.	Supports inheritance , allowing code reuse from existing classes.
Overloading	Does not support function or operator overloading.	Supports polymorphism through function and operator overloading.
Data Movement	Data flows freely around the system from function to function.	Objects communicate with each other through member functions .

Problem Handling	Best suited for small to medium, less complex programs.	Designed to handle large, complex , and real-world applications.
Examples	C, Pascal, FORTRAN, BASIC.	Java, Python, C++, C#, Ruby.

Advantages of OOP over POP

1. Better Data Security -
2. Code reusability -
3. Modularity - Programs are divided into objects and classes, making the system easier to manage. POP organizes programs mainly around functions, which can become complex.
4. Improved Scalability - OOP is suitable for large and complex applications. POP is generally better suited for small programs.
5. Easier Maintenance -
6. Reduced Code Duplication -
7. Better Testing Facilities -
8. Improved Collaboration –
9. Greater Flexibility and Extensibility -
10. Better Real-World Modeling of software -

Feature	Compile-Time Polymorphism	Run-Time Polymorphism
Alternative Names	Static Binding, Early Binding	Dynamic Binding, Late Binding
Implemented Via	Overloading (Method/Operator)	Overriding
Resolved By	Compiler	JVM / Runtime System
Execution Speed	Fast	Comparatively Slower
Inheritance	Not required	Mandatory
Flexibility	Low (Fixed at compile time)	High (Dynamic at runtime)
Example	fun(int) or fun(double)	Animal.sound() or Bird.sound()

Compile Time polymorphism	
<pre> class MathWizard { int add(int a, int b) { return a + b; } int add(int a, int b, int c) { return a + b + c; } double add(double a, double b) </pre>	<pre> public class Main { public static void main(String[] args) { MathWizard mw = new MathWizard(); System.out.println("Overloading 1 (2 ints): " + mw.add(5, 10)); System.out.println("Overloading 2 (3 ints): " + mw.add(5, 10, 15)); } } </pre>

<pre>{ return a + b; }</pre>	
--	--

Run Time polymorphism	
<pre>class Animal { void makeSound() { System.out.println("The animal makes a sound"); } } class Dog extends Animal { @Override void makeSound() { System.out.println("The dog barks: Woof! Woof!"); } }</pre>	<pre>public class Main { public static void main(String[] args) { Animal myAnimal = new Animal(); Animal myDog = new Dog(); myAnimal.makeSound(); myDog.makeSound(); } }</pre>